Manus login for games

Use this page when an approved Game Blueprint or an existing game needs Manus player login.
Keep offline games offline and retain `applicationKind: game`. The shared authentication contract owns OAuth, session cookies and Preview transport; payments owns Stripe. Godot and Three.js reuse the same Webdev implementation.

Enable and install

GET the current configuration, then enable `features.server` and `features.database` through its current revision. New Games initialize with both off; there is no `features.auth` flag.
The canonical Webdev backend needs a database for its `users` table. Infrastructure enablement does
not install application login or apply migrations.

From a project with no backend, run the packaged installer from the runtime directory returned by
init/attach:

- Godot: `node <runtime-directory>/install-auth.mjs`
- Three.js: `node <runtime-directory>/install-auth.mjs --threejs`

Both install the same `server/` containing Webdev's OAuth callback, user repository, session context,
auth router and protected procedures. The build imports these owners directly from `web-db-user-v1`;
there is no separate Game OAuth handler, JWT format or user identity store. Server credentials remain
in the injected runtime environment. The installer never runs DDL, provisions services or deploys.

Merge the returned `configurationPatch`, preserving unrelated fields and routes, then verify its stored and active state. Both declare server/database, `server/Dockerfile`, `/api/health`, `/api/*` server routing and the final static fallback. Godot keeps its `site` build.
Three.js keeps `pnpm build` → `dist` without
Godot files or a Godot release contract. Frozen starters are unchanged.

Install server dependencies and commit `server/package-lock.json`; the Dockerfile uses `npm ci`.
For Three.js, install the added browser dependencies and commit the project lockfile too.
Apply
`server/0000_initial_users.sql` through the normal project migration workflow, not to Addon DB

or
blindly over an incompatible existing users table. Preserve users, player IDs and game data. The Dashboard's Database page reads the project's real database binding, not a fabricated Preview
user store; verify that binding before interpreting Preview rows.

Shared browser login and player APIs

Both engines use the same `ManusAuth` browser bridge, built from Webdev `startLogin` and its tRPC
client. Call `prepare()` during startup and use `get_session()` or `manus-auth-change` for player UI. OAuth callback, state-cookie and session validation follow the shared
authentication contract. Startup reads the real session;
a redirect alone is not login success. The installed `/api/oauth/callback` upserts the canonical user,
sets `webdev_app_session` and redirects to `/`.

Call `login()` directly from the button/input handler. Embedded login opens the standalone game
synchronously while the click has user activation; that game's startup continues login. Standalone
login navigates directly to the provider. Call `logout()` to clear the canonical session cookie and
player UI. Browser cookie restrictions can prevent the embedded view from sharing the standalone
session; do not fabricate an authenticated iframe state.

Add gameplay reads and writes in `server/routers.mjs` using Webdev `protectedProcedure`. Derive
ownership from `ctx.user`, never input user IDs. Use `ManusAuth.query('procedure', input)` and `ManusAuth.mutate('procedure', input)` with the application's actual procedure names. These use JSON tRPC and application cookies. Keep the server's JSON-only mutation guard and CORS disabled;
form/text submissions must not mutate data through a cross-site browser. Do not restore the retired
Game `/api/auth/login`, `/api/auth/session`, `auth.authenticate(req)` or custom CSRF-token protocol.

In Three.js `src/main.ts`, use `import { ManusAuth } from '../manus-auth.js'`; the installer also supplies `manus-auth.d.ts` for the starter's strict TypeScript build. Development uses the Web runtime environment and a Vite `/api` proxy; saved Preview and publication use the normal

Web hybrid deployment. Do not apply Godot PCK, loader or resident-preview instructions to Three.js.

Godot browser bridge

The helper runs in the authored HTML shell, not inside PCK. The installer injects the self-contained
browser bundle into `web/loading.html`, gates `engine.startGame()` on `ManusAuth.startGame(...)`, and
adds `scripts/manus_auth.gd` as an autoload. The JS/GDScript bridge only transports login/logout,
session state and errors; it does not implement another OAuth protocol or exchange codes in `_ready`.

Inspect `html/custom_html_shell` in `export_presets.cfg`. The installer preserves unsupported custom
loaders and reports the authored path. For Scroller, adapt `web/loader.js` and regenerate `web/shell.html` through `web/build-shell.mjs`; Puzzle uses `ui/web/shell.html`. Wire the packaged browser bridge and await `ManusAuth.startGame(() => engine.startGame(...))` in that authored startup.
Report startup/session failures through the existing loader UI.

Godot exposes `ManusAuth.login()`, `logout()`, `get_session()`, `open_standalone()`, `auth_state_changed`
and `auth_error`. Invoke login or standalone opening directly in the input callback, without deferring
or awaiting other work first. `auth_bootstrap_missing` means the browser bridge was not loaded.
Game-specific API calls can use the browser bridge's `query` / `mutate`; keep their business logic in
protected server procedures.

Resident Godot Preview and saved-version history have no writable player backend. Their explicit
`online_preview_requires_checkpoint` response starts the game offline. Login, logout and protected
writes remain unavailable there. Use a verified standalone online checkpoint Preview or published
hybrid game for real login and writes; do not widen saved-history API relays. A checkpoint or health
200 is not evidence that an online backend, cookie or database binding works.

Stripe and migration

For payments, follow Stripe Checkout and buyer identity
and webhook processing. Register the webhook outside player tRPC routes,
without player session or Origin checks; Stripe signature verification authenticates it. Preserve existing
order, entitlement and leaderboard logic.

Existing backends, loaders and dependencies are never silently overwritten. For an existing game,
adapt its client calls and server routes together to the canonical Webdev sources, preserve its
user database and secrets, then remove the superseded Game auth handler/bridge protocol. Installing
a new Addon package alone does not migrate an existing game's copied code.

Verify callback success and rejection, session readback, refresh, logout, two-account isolation and
protected gameplay/Checkout writes on the supported proxy paths. Legitimate requests must succeed
when a proxy rewrites Origin; anonymous and cross-site requests must remain rejected. Verify Webhook
signatures and fulfillment separately. Record Cloud, Work Locally, online checkpoint and published
coverage accurately; local doubles are not hosted OAuth/Stripe acceptance. Never log codes, tokens,
cookies or raw callback URLs.